



Universidad de Caldas

ESPECIFICACIONES DE ENTREGABLES

Proyecto K-QGMIP: Manual de Usuario

Análisis y Diseño de Algoritmos

Facultad de Inteligencia Artificial e Ingenierías

2026-1

Introducción

Como parte integral del proyecto K-QGMIP, cada equipo debe entregar documentación técnica completa que permita comprender el trabajo desarrollado. En particular el Manual de Usuario, está orientado a usuarios finales que necesitan operar el software de forma rápida siguiendo un orden en la ejecución.

La calidad de la documentación es un criterio de evaluación fundamental, ya que refleja la profundidad de comprensión del problema, la capacidad de comunicar las estrategias complejas de manera efectiva, y la claridad con la que se aborda el desarrollo de software. Un buen proyecto se refuerza con documentación clara, completa y efectiva que potencia significativamente el impacto del trabajo realizado.

Este documento especifica en detalle los requisitos, estructura y contenido que deben tener el manual de usuario.

Convenciones de Nomenclatura

Para mantener consistencia y facilitar la organización del código, se establecen las siguientes convenciones de nomenclatura para los repositorios y carpetas del proyecto:

Estrategia	Nombre de Repositorio/Carpeta
GeoMIP K-particiones	KGeoMIP
QNodes K-particiones	KQNodes

Estas convenciones deben aplicarse consistentemente en:

1. Nombre del repositorio Git
2. Carpeta principal del proyecto
3. Nombre de la clase principal que implementa la estrategia
4. Referencias en documentación y presentaciones

La 'K' inicial hace referencia a 'k-particiones', distinguiendo claramente estas extensiones de las implementaciones originales de bi-particiones (GeoMIP y QNodes).

Manual de Usuario

1. Propósito

El Manual de Usuario está dirigido a usuarios finales que necesitan utilizar el software desarrollado sin entrar en los detalles algorítmicos internos. Este documento debe

permitir a un usuario instalar, configurar y utilizar el software exitosamente para analizar los sistemas propios.

2. Estructura y Contenido Requerido

2.1 Introducción y Visión General

Presentación accesible del software y sus capacidades:

5. **Qué hace el software:** Explicación en lenguaje sencillo cual es la funcionalidad principal (encontrar k-particiones óptimas de sistemas).
6. **Para qué sirve:** Aplicaciones prácticas y casos de uso típicos.
7. **Conceptos básicos:** Explicación intuitiva de qué es una k-partición y qué significa 'partición de mínima información', sin matemáticas avanzadas.
8. **Capacidades y limitaciones:** Qué puede y qué no puede hacer el software, tamaños de sistema que puede manejar razonablemente.

2.2 Requisitos del Sistema

Especificaciones técnicas mínimas y recomendadas:

9. **Sistema operativo:** Versiones de Windows, macOS o Linux soportadas.
10. **Hardware:** Procesador (velocidad y núcleos recomendados), memoria RAM (mínima y recomendada según tamaño de sistema a analizar), y espacio en disco.
11. **Software:** Versión de Python requerida, bibliotecas necesarias con versiones específicas, y herramientas adicionales si las hay.

2.3 Instalación Paso a Paso

Guía detallada y secuencial para instalar el software:

12. **Descarga del proyecto:** Dónde obtener el código fuente (repositorio Git, archivo comprimido, etc.), instrucciones específicas de descarga.
13. **Instalación de dependencias:** Comandos exactos para instalar Python y bibliotecas necesarias, manejo de entornos virtuales, y solución a problemas comunes de instalación.
14. **Configuración inicial:** Archivos de configuración que deben modificarse, variables de entorno necesarias, y verificación de instalación correcta.
15. **Capturas de pantalla:** Imágenes mostrando cada paso del proceso de instalación, especialmente en puntos donde usuarios suelen tener dudas.

2.4 Video Tutorial de Instalación y Uso

Cada equipo debe producir un video demostrativo que facilite la comprensión del proceso de instalación y uso básico del software.

Características del video:

16. **Duración:** Entre 8 y 15 minutos. Debe ser suficientemente detallado pero conciso.
17. **Contenido obligatorio:**
 1. Proceso completo de instalación desde cero en un sistema limpio
 2. Configuración inicial del entorno
 3. Ejecución de al menos un ejemplo completo mostrando:
 - Preparación de datos de entrada
 - Ejecución del programa con diferentes valores de k
 - Interpretación de los resultados obtenidos
 4. Visualización de resultados
18. **Calidad técnica:**
 1. Captura de pantalla clara y legible (les dejo esta que es una buena resolución, ojalá mínimo 1280x720)
 2. Audio claro si se incluye narración en vivo, o
 3. Subtítulos en español y deben sincronizarse correctamente con el contenido (si no hay narración)
 4. Zoom apropiado en secciones donde se muestra código o comandos
 5. Edición básica para eliminar tiempos muertos o errores
19. **Formato de entrega:**
 1. Entregar el archivo de video en formato MP4
20. **Herramientas sugeridas:** OBS Studio (gratuito), Camtasia, ScreenFlow, o herramientas similares de captura y edición de pantalla. Para subtítulos: YouTube Studio o herramientas de subtitulado como Subtitle Edit. Esas son algunas herramientas que les sugiero, pero ustedes pueden usar las herramientas que mejor les parezcan.

OJO: El video es un complemento esencial al manual escrito. Muchos usuarios aprenden mejor viendo el proceso en acción que leyendo instrucciones. Un video bien producido es una gran ayuda para lograr usar el software exitosamente de forma rápida.

2.5 Guía de Uso Básico

Instrucciones para realizar las operaciones más comunes:

21. **Preparación de datos de entrada:** Formato exacto requerido para especificar sistemas (archivos de texto, JSON, matrices, etc.), ejemplos de archivos de entrada válidos, y herramientas para generar o convertir datos.
22. **Ejecución básica:** Comando o interfaz para ejecutar el análisis, parámetros básicos necesarios (valor de k, sistema a analizar), y ejemplo completo de ejecución desde inicio hasta fin.
23. **Interpretación de resultados:** Qué información produce el software, cómo leer la salida (k-partición encontrada, valor de pérdida, etc.), y ejemplos de salidas típicas explicadas.
24. **Casos de uso típicos:** Escenarios comunes paso a paso (encontrar 3-partición de sistema pequeño, comparar particiones para diferentes k, etc.).

2.6 Opciones y Parámetros Avanzados

Configuraciones opcionales para usuarios experimentados:

25. **Parámetros de configuración:** Lista completa de parámetros ajustables, descripción de qué controla cada parámetro, valores por defecto y rangos recomendados.
26. **Modos de operación:** Si el software tiene múltiples modos (búsqueda exhaustiva, heurística, modo debug, etc.), explicar cada uno y cuándo usarlo.
27. **Opciones de salida:** Formatos de salida disponibles, nivel de detalle configurable, y opciones de visualización.
28. **Optimización de rendimiento:** Consejos para ajustar parámetros según recursos disponibles, trade-offs entre precisión y velocidad.

2.7 Solución de Problemas

Diagnóstico y resolución de problemas comunes:

29. **Errores comunes:** Lista de mensajes de error frecuentes, qué significa cada error, y cómo solucionarlo.
30. **Problemas de instalación:** Dependencias faltantes, conflictos de versiones, y problemas específicos por sistema operativo.
31. **Problemas de ejecución:** El programa no inicia, se cuelga durante ejecución, consume memoria excesiva, produce resultados inesperados.
32. **Datos de entrada problemáticos:** Validación de formato, detección de errores en especificación del sistema, y ejemplos de corrección.

2.8 Ejemplos y Tutoriales

Casos prácticos completos que ilustran el uso del software:

33. **Tutorial básico:** Análisis paso a paso de un sistema simple (3-4 nodos), incluyendo preparación de datos, ejecución, e interpretación de resultados.
34. **Caso de estudio intermedio:** Sistema de tamaño moderado (8-10 nodos), explorando diferentes valores de k y comparando resultados.
35. **Ejemplo avanzado:** Uso de parámetros avanzados, optimización de rendimiento para sistema grande, y análisis detallado de resultados.

2.9 Referencia Rápida

Material de consulta rápida, si es del caso:

36. **Comandos principales:** Lista de comandos con sintaxis y breve descripción.
37. **Tabla de parámetros:** Referencia tabular de todos los parámetros con valores por defecto y rangos válidos.
38. **Formato de archivos:** Especificación técnica concisa de formatos de entrada y salida.
39. **Glosario:** Definiciones breves de términos técnicos utilizados en el manual.

2.10 Características de Formato y Presentación

El Manual de Usuario debe cumplir con los siguientes estándares de formato:

40. **Extensión:** Debe ser suficientemente completo pero conciso.
41. **Formato:** Documento word, tamaño carta, fuente Arial o Calibri de 11 puntos.
42. **Lenguaje:** Claro y accesible, evitando jerga técnica innecesaria. Cuando se usen términos técnicos, definirlos la primera vez.
43. **Capturas de pantalla:** Abundantes y de alta calidad. Marcar elementos relevantes con anotaciones cuando sea necesario.
44. **Ejemplos de código:** Usar fuente monoespaciada, con comentarios explicativos. Limitar longitud de ejemplos a lo esencial.
45. **Organización visual:** Uso de cajas de texto, íconos, o colores para destacar advertencias, notas importantes, o tips.
46. **Navegación:** Tabla de contenidos con hipervínculos, índice si es extenso, y referencias cruzadas claras entre secciones.
47. **Video tutorial:** Enlace prominente al video en las primeras páginas del manual, idealmente en la sección de instalación.
48. **Usabilidad:** El manual debe ser completamente auto-contenido. La idea es que no debo necesitar buscar información adicional para operaciones básicas.

Criterios de Evaluación de la Documentación

La documentación (Manual Técnico y Manual de Usuario) representa una porción significativa de la evaluación del proyecto. A continuación se detallan los criterios específicos que se utilizarán para evaluar cada manual.

3.1 Evaluación del Manual Técnico (40% de la nota de documentación)

49. **Rigor matemático y claridad conceptual:** Precisión en definiciones, corrección de formulaciones matemáticas, claridad en explicaciones de conceptos complejos.
50. **Calidad de arquitectura y diagramas:** Completitud de diagramas UML, corrección en notación, claridad visual, y utilidad para comprender la estructura del código.
51. **Calidad de la descripción algorítmica:** Completitud del pseudocódigo, claridad en explicación de decisiones de diseño, facilidad de reproducción del algoritmo.
52. **Análisis de complejidad:** Corrección del análisis teórico, identificación precisa de cuellos de botella, validación empírica del análisis.
53. **Resultados experimentales:** Comprensividad de experimentos, calidad de visualizaciones, profundidad del análisis e interpretación.
54. **Reflexión crítica:** Honestidad sobre limitaciones, identificación de mejoras potenciales, demostración de comprensión profunda.

3.2 Evaluación del Manual de Usuario (30% de la nota de documentación)

- 55. **Claridad y accesibilidad (25%):** Lenguaje apropiado para audiencia no técnica, explicaciones intuitivas de conceptos complejos, evita jerga innecesaria.
- 56. **Compleitud de instrucciones (25%):** Cobertura de todos los pasos necesarios, detalle suficiente para reproducir operaciones, anticipación de problemas comunes.
- 57. **Calidad del video tutorial (20%):** Claridad de grabación, completitud del contenido, efectividad pedagógica, calidad de subtítulos (si aplica).
- 58. **Calidad de ejemplos y tutoriales (15%):** Casos de uso realistas y relevantes, claridad en presentación paso a paso, utilidad pedagógica.
- 59. **Material de soporte (10%):** Calidad y cantidad de capturas de pantalla, diagramas de flujo útiles, sección de troubleshooting efectiva.
- 60. **Usabilidad del documento (5%):** Organización lógica, facilidad de navegación, índice y tabla de contenidos útiles.

3.3 Aspectos Transversales (30% de la nota de documentación)

Criterios que aplican a ambos manuales:

- 61. **Calidad de redacción (40%):** Gramática y ortografía impecables, coherencia y fluidez en la argumentación, estructura lógica de ideas.
- 62. **Calidad de presentación (30%):** Formato profesional y consistente, figuras y tablas de alta calidad, cumplimiento de especificaciones de formato.
- 63. **Compleitud (20%):** Cobertura de todos los aspectos requeridos, ausencia de secciones incompletas o placeholder text.
- 64. **Profesionalismo (10%):** Nivel de detalle apropiado, balance entre exhaustividad y concisión, citación apropiada de fuentes.

4. Recomendaciones Finales

4.1 Proceso de Desarrollo de la Documentación

Se recomienda fuertemente desarrollar la documentación de manera iterativa y paralela al código, no como una actividad de última hora. Escribir explicaciones de algoritmos y decisiones de diseño mientras se está implementando ayuda a clarificar el propio pensamiento y a detectar problemas tempranamente.

Dedicar tiempo específico cada semana a documentación. Una buena regla empírica es que por cada hora de codificación, se debe invertir al menos 30 minutos en documentación. Esto resulta en mejor código y documentación de mayor calidad con menor esfuerzo total al final del proyecto.

4.2 Revisión y Refinamiento

Antes de la entrega final, realizar múltiples rondas de revisión:

65. **Revisión técnica:** Verificar corrección de todas las afirmaciones matemáticas y algorítmicas. Validar que pseudocódigo sea reproducible.
66. **Revisión de usabilidad:** Idealmente, pedir a alguien ajeno al equipo que siga el Manual de Usuario para validar que las instrucciones son claras y completas.
67. **Revisión de diagramas:** Verificar que todos los diagramas UML sigan notación estándar, sean legibles, y correspondan fielmente al código implementado.
68. **Revisión del video:** Ver el video completo varias veces, verificar sincronización de subtítulos, y asegurar que todos los pasos sean claros y reproducibles.
69. **Revisión de estilo:** Corrección exhaustiva de gramática, ortografía y estilo. Verificar consistencia en terminología a lo largo de todo el documento.
70. **Revisión de formato:** Verificar que figuras estén numeradas correctamente, referencias cruzadas funcionen, tabla de contenidos esté actualizada.

4.3 Recursos y Herramientas

Para facilitar la creación de documentación de alta calidad, se recomiendan las siguientes herramientas:

71. **Para ecuaciones:** LaTeX (Overleaf online), MathType, o editor de ecuaciones de Word.
72. **Para diagramas UML:** Draw.io, Lucidchart, PlantUML, Visual Paradigm, o StarUML.
73. **Para gráficas:** Matplotlib/Seaborn (Python), ggplot2 (R), o herramientas de visualización interactiva.
74. **Para capturas de pantalla:** Lightshot, Snagit, Greenshot, o herramientas nativas del sistema operativo con capacidades de anotación.
75. **Para video:** OBS Studio (gratuito), Camtasia, ScreenFlow, o Loom. Para edición: DaVinci Resolve (gratuito), Adobe Premiere, o iMovie.
76. **Para subtítulos:** YouTube Studio (automático), Subtitle Edit, Aegisub, o herramientas online como Kapwing.
77. **Para control de versiones:** Usar Git no solo para código sino también para documentación, permitiendo rastrear cambios y colaborar efectivamente.

4.4 Convenciones de Nomenclatura en la Documentación

Asegurarse de usar consistentemente los nombres **KGeoMIP** y **KQNodes** a lo largo de toda la documentación, diagramas, código, y video tutorial. Esta consistencia facilita la comprensión y navegación del proyecto.

4.5 Contacto y Consultas

Si durante el desarrollo de la documentación surgen dudas sobre qué incluir, nivel de detalle apropiado, o interpretación de estos requisitos, los equipos deben consultar con el profesor de la asignatura. Es preferible aclarar dudas tempranamente que entregar documentación que no cumple con las expectativas.

Estas especificaciones son detalladas pero no rígidas. Se valora la iniciativa y creatividad en la presentación de la información, siempre que se cubran todos los aspectos requeridos y se mantenga claridad y rigor apropiados.

La documentación de calidad es una habilidad profesional fundamental. El esfuerzo invertido en desarrollar estos manuales no solo contribuye a la evaluación del proyecto, sino que representa práctica valiosa en comunicación técnica que será esencial en la carrera profesional futura.